

الشرح التقني الشامل لمشروع Smart Hotel Booking (للجامعة)

هذا الملف يشرح كيفية عمل المشروع تقنياً من البداية (Frontend) إلى النهاية (Database) لكل ميزة، بناءً على التنفيذ الحقيقي في الكود، مع تبيان الملفات التي يجب فتحها أثناء المناقشة.

1. إدارة المستخدمين واستعادة كلمة المرور (User Management & Password Recovery)

ماذا يفعل المستخدم؟

يقوم بالتسجيل وإدخال البيانات، ويختار سؤالين أمنيين (باللغة الإنجليزية) ويجب عليهما. في حال نسيان كلمة المرور، يكتب بريده، فتظهر له الأسئلة نفسها، ليحجب عليها ويغير كلمة المرور. (لا يوجد إيميل أو SMS).

مسار البيانات (Data Flow - Password Recovery)

- Frontend:** المستخدم يضغط "Forgot Password" في `frontend/src/pages/Login.jsx`.
- Action:** يدخل البريد، فيتم استدعاء الدالة `handleForgotPassword`.
- API Request:** يرسل ال Frontend طلب `POST /api/auth/forgot-password` ويحتوي فقط `{ email }`.
- Backend Route:** يستقبله ال `authRoutes.js`.
- Controller:** يستقبله `authController.js` (دالة `forgotPassword`).
- Model & Database:** يقوم بالبحث عبر `User.findOne` في جدول `users`.
- Response:** يعيد ال Backend الأسئلة الأمنية (بدون الإجابات).
- Frontend:** تعرض `Login.jsx` الأسئلة للمستخدم ليحجب عليها.
- API Request 2:** عند الإرسال، يرسل `POST /api/auth/verify-answers`.
- Controller:** دالة `verifySecurityAnswers` تقوم بمطابقة الإجابات. إذا نجحت، يعود Token للاستعادة.
- Frontend:** يظهر فورم لتغيير كلمة المرور.

الملفات التي أفتحها أثناء المناقشة:

- `backend/src/controllers/authController.js` (دوال `forgotPassword` و `verifySecurityAnswers`).

2. مدير النظام وإنشاء حسابات الفنادق (Admin & Hotel Manager)

المعمارية الحقيقية للمشروع:

لا يوجد نظام "اختيار فندق وربطه بمدير". المعمارية المعتمدة هي أن المدير يمثل الفندق. عندما ينشئ الأدمن حساب مدير، يتم إنشاء الفندق برمجياً وبشكل آلي في نفس اللحظة. وعند حذف حساب المدير، يتم حذف الفندق وكل ما يرتبط به. ال Admin لا يتدخل في إدارة الغرف أو الأسعار، بل هذا دور المدير من لوحة تحكمه الخاصة.

مسار البيانات (Data Flow - Create Manager)

1. **Frontend**: الأدمن يدخل بيانات المدير (الاسم، الإيميل، الباسورد) ويضغط Submit في `AdminPortal.jsx`.
2. **API Request**: يرسل `POST /api/admin/users`.
3. **Controller**: `adminController.js` يستقبل الطلب في دالة `createUser`.
4. **Transaction**: يفتح `sequelize.transaction` لضمان التزامن وحماية البيانات.
5. **Hotel Creation**: ينشئ الفندق برمجياً `Hotel.create` باسم مؤقت وصورة عشوائية (random img id) من 1 لـ 150.
6. **User Creation**: ينشئ حساب المدير `User.create` ويضع داخله `hotel\_id` الذي تم إنشاؤه للتو.
7. **Commit**: يعتمد ال Transaction. إذا فشل أي أمر، يتراجع عن كل شيء لضمان عدم وجود فندق بلا مدير أو مدير بلا فندق.

مسار البيانات (Data Flow - Delete Manager)

1. **Frontend**: يضغط الأدمن أيقونة الحذف في `AdminPortal.jsx`.
2. **API Request**: `DELETE /api/admin/users/:id`.
3. **Controller**: دالة `deleteUser`.
4. **Cleanup**: يقوم ال Backend بحذف القیود المتعلقة بالفندق (Bookings, Favorites, LoyaltyTransaction).
5. **Deletion**: يحذف حساب المدير، ثم يحذف الفندق `hotel.destroy`.

الملفات التي أفتحها أثناء المناقشة:

- `backend/src/controllers/adminController.js` (دوال `createUser` و `deleteUser` وال Transactions).

3. البحث الذكي ونظام التوصية (Search & Recommendation)

ماذا يفعل المستخدم؟

يبحث عن فندق ويحدد المدينة، وتفضيلات الرحلة (ميزانية، عائلة/عمل، الخ). يظهر له فندق كـ "مقترح" مع نسبة مطابقة (Match Percentage).

مسار البيانات والخوارزمية (Data Flow & Algorithm)

1. **Frontend:** `Hotels.jsx` يأخذ البيانات ويرسلها لـ ``api.get('/recommendations', { params })``.
2. **Controller:** ``recommendationController.js`` (دالة ``getRecommendations``).
3. **Data Fetching:** يجلب فقط الفنادق الموجودة في المدينة المحددة (``city_id``). (المدينة ليست لها وزن في الخوارزمية لأنها تستخدم للتصفية الأساسية أولاً).
4. **Scoring Loop:** يمر على كل فندق ويحسب ``Score`` يبدأ من 0:
 - التقييم (**Rating**): النجوم مضروبة في 15.
 - السعر (**Budget**): معادلة مئوية تحسب الفرق بين سعر الفندق والسعر المستهدف. اقتراب السعر يزيد النقاط.
 - الخدمات (**Amenities**): مطابقة كل خدمة مطلوبة تعطي 12 نقطة.
 - نوع الرحلة (**Trip Type**): إذا كان نوع الرحلة ``business`` والفندق يمتلك واي فاي أو غرفة اجتماعات، تزيد النقاط 25.
 - العروض (**Flash Deals**): وجود عرض يعطي 25 نقطة.
5. **Normalization & Response:** يتم قص الـ ``Score`` ليكون الحد الأقصى 99 (``Math.min(99, score)``). يتم فرز المصفوفة تنازلياً وإعادة ترتيبها.
6. **Frontend UI:** يعرض النسبة المئوية ``% Match`` ويعرض أول سبب مطابق (مثلاً: "Ideal for Couples").

الملفات التي أفتحها أثناء المناقشة:

- ``backend/src/controllers/recommendationController.js`` (شرح كيفية تجميع الـ ``Score`` والمعادلة الرياضية).

4. مقارنة المنافسين في لوحة المدير (Competitor)

ماذا يفعل المستخدم؟

مدير الفندق يفتح تبويب Benchmarking في `ManagerPortal.jsx` ليرى أداء فندقه مقارنة بالسوق.

مسار البيانات (Data Flow)

1. **API Request:** يرسل `GET /api/manager/competitor-benchmarking`.
2. **Controller:** `managerController.js` دالة `getCompetitorBenchmarking`.
3. **Data Privacy (أهم نقطة للمناقشة):**
 - يجلب الـ Backend جميع الفنادق في نفس الـ `city\_id` عبر `Hotel.findAll`.
 - يقوم بتصفية (**Filter**) فندق المدير من القائمة لكي لا يقارن الفندق مع نفسه.
 - يحسب المجموع الحسابي (`reduce`) لأسعار المنافسين وتقييماتهم.
 - يعيد المتوسط فقط `marketAvgPrice` للـ Frontend.
 - النتيجة: لا يتم كشف أسعار الفنادق المنافسة بشكل فردي أبداً، مما يحمي خصوصية البيانات ويفي بالمتطلب الجامعي.

الملفات التي أفتحها أثناء المناقشة:

- `backend/src/controllers/managerController.js` (دالة `getCompetitorBenchmarking`).

5. نظام الولاء المعزول (Isolated Loyalty System)

ماذا يفعل المستخدم؟

يحجز فندقاً فيحصل على نقاط تخص هذا الفندق بالتحديد. يحاول استخدام النقاط في حجز فندق آخر فلا يستطيع.

مسار البيانات والخوارزمية (Data Flow)

1. **التخزين (Database):** يستخدم جدول `user\_loyalty` بحقول `(user\_id, hotel\_id, current\_points)` أي أن رصيد النقاط معزول تماماً لكل فندق.
2. **الاستبدال (Redeem) أثناء الحجز:**
 - يختار المستخدم Voucher/Reward. يتم إرسال الـ `reward\_id` في طلب إنشاء الحجز `createBooking`.

- في `bookingController.js`، يبحث ال Backend في الرصيد الخاص به: ``javascript
 `` UserLoyalty.findOne({ where: { user_id, hotel_id } })
- إذا كان الرصيد كافياً، يخصم ال Backend قيمة المكافأة مالياً من السعر الكلي، ويخصم النقاط من `current\_points`.

3. إضافة النقاط (Earn):

- في نهاية كود الحجز الناجح، يحسب ال Backend النقاط المكتسبة (السعر الكلي × معامل النسبة الثابت).
- يتم تحديث جدول `user\_loyalty`، وتسجيل حركة في `loyalty\_transactions`.

الملفات التي أفتحتها أثناء المناقشة:

- `backend/src/models/UserLoyalty.js` (لإظهار أنه مرتبط بحقل ال `hotel\_id`).
- `backend/src/controllers/bookingController.js` (شرح خصم واكتساب النقاط في دورة الحجز).

6. الخرائط والأماكن السياحية (Maps & Nearby Services)

المسار (Data Flow)

1. **Frontend**: مكون `HotelDetail.jsx` يستخدم مكتبة `react-leaflet`.
2. **External API**: يرسل طلب GET صامت إلى خوادم `Overpass API` (جزء من OpenStreetMap) تحتوي إحداثيات الفندق `latitude` و `longitude`.
3. **Retrieval**: يسترد ال API الأماكن القريبة بقطر محدد.
4. **Calculations**: باستخدام خوارزمية **Haversine** المبرمجة محلياً في ال Component: `getDistanceFromLatLonInKm(lat1, lon1, lat2, lon2)` يتم حساب المسافة الدقيقة لكل مطعم أو صيدلية أو مكان سياحي.
5. **UI Update**: تعرض الخريطة ال Markers وتظهر قائمة الأماكن مع مسافتها.

الملفات التي أفتحتها أثناء المناقشة:

- `frontend/src/pages/HotelDetail.jsx` (دالة ال Haversine واستعلام Overpass الخارجي).

7. مقارنة الفنادق وتصدير PDF (Hotel Comparison)

المعالجة والمخرجات (Data Flow)

1. يختار المستخدم فنادق، تُحفظ الأرقام المعرفة (IDs) في React State.
2. **API Request**: يرسلها إلى `POST /api/comparison/matrix``.
3. **Controller**: `comparisonController.js`` يستقبلها ويبنى (Amenity Matrix).
 - **التحدي**: ماذا لو كان فندق لا يقدم خدمة الواي فاي بينما يقدمها الباقي؟
 - **الحل**: الخادم ينشئ مصفوفة شاملة بكل الخدمات، وإذا فقد الفندق خدمة، يضيفها الخادم مع قيمة `available: false``.
4. **PDF Export**: زر التصدير في `Compare.jsx`` يستخدم مكتبة `html2pdf.js`` لأخذ ال DOM Element للجدول المعروض، تحويله ل Canvas، وحفظه ك PDF دون الحاجة لمعالجة في ال Backend.

8. نظام الحجوزات والأسعار الديناميكية (Booking & Dynamic Pricing)

ماذا يحدث عند إنشاء الحجز؟

1. **Transaction & Locks**: يبدأ `sequelize.transaction()`` لحماية الحجز من الانقطاع.
2. **Room Availability**: يتأكد الكود من توفر الغرف `room.available_rooms < numRooms`` وإذا لم يتوفر يفشل الحجز.
3. **Dynamic Pricing**: يتأكد إن كان مدير الفندق قد وضع `DynamicPricingRule`` لهذا التاريخ. إذا وُجدت، يُضرب السعر بالمعامل المحدد (`multiplier``).
4. **Flash Deals**: إن كان هناك عرض فعال `Flash Deal``، يخصم الكود قيمته.
5. **Loyalty Deductions**: تُخصم المكافآت المالية من نظام الولاء إن طُلبت.
6. **Taxes**: يُحسب 3% ضرائب كإجمالي `tax_amount``.
7. **Commit**: يتم إنقاص الغرف `available_rooms`` فعلياً وحفظ الحجز والتسجيل.

الملفات التي أفتحها أثناء المناقشة:

- `backend/src/controllers/bookingController.js`` (دالة `createBooking`` لفهم تعقيدات التسعير).

كيف أشرح سير العمل العام أثناء ال Docker Run؟

عند كتابة أمر `docker compose up`:

1. **SQL Server**: تعمل حاوية (Container) قاعدة البيانات أولاً على البورت 1433 وتتجهز للاتصالات.
2. **Backend (Node.js)**: تتصل بقاعدة البيانات. يقوم ال (Sequelize) ORM بإنشاء الجداول المفقودة عبر `.sync()`.
3. **Frontend (Vite/Nginx)**: تعمل حاوية الواجهة. الطلبات تخرج من واجهة المتصفح، تذهب عبر ال API إلى Backend Container على البورت 5000 (أو البورت المحدد)، والذي بدوره يحادث قاعدة البيانات ويعيد ال JSON Response.